

Indice

ThreatForge MR comparison - current vs proposed	1
MR-0001	1
Current document	1
Proposed rewrite for metamodel test	2
What moved down	3
MR-0002	3
Current document	3
Proposed rewrite for metamodel test	4
What moved down	5
MR-0003	5
Current document	5
Proposed rewrite for metamodel test	6
What moved down in the first reformulation	6
Second reformulation after ADR-family review	7
Why the first reformulation was reopened	7
MR-0004	8
Current document	8
Proposed rewrite for metamodel test	9
What moved down	9
MR-0005	9
Current document	9
Proposed rewrite for metamodel test	10
What moved down in the first reformulation	11
Second reformulation after ADR-family review	11
Why the first reformulation was reopened	12
Lesson learned from MR-0003 versus MR-0005	12
Observation	12
Evidence that preserved the distinction for further testing	12
Questions that must be asked before merging or splitting Macro Requirements	12
Current disposition	13
General lesson for the metamodel	13

ThreatForge MR comparison - current vs proposed

Non-canonical case study. Baseline for current documents is the pinned ThreatForge product-semantic commit below.

`cae0f7b6b37f430ac4e857aabf6ef9f87c89dbb1`

The purpose is to test the general MR metamodel against a real project and to collect refactoring/authoring evidence. The proposed rewrites are **not canonical ThreatForge changes**.

MR-0001

Current document

Original: MR-0001 - Gestione documentale governata

Intent Define a governed documentation system that gives ThreatForge one concise, analyzable and navigable source for project intent, decisions, requirements, assets, relations and verification evidence.

Context ThreatForge treats governed documentation as the primary project model for people, developers, LLM consumers and deterministic checks. Canonical registries and Markdown bodies provide the maintained sources, while readable books, HTML views, diagrams, indexes, appendices and reports are derived projections.

The model organizes project knowledge by Macro-requirement and uses explicit Decisions and Requirements to refine each project theme. Controlled vocabularies, taxonomies, stable identifiers and explicit relations reduce ambiguity and support future security and threat analysis.

Macro obligation

- ThreatForge must organize governed project knowledge around stable Macro-requirement identities.
- Each Macro-requirement must provide the project-level context for its owned Decisions and Requirements.
- Governed documentation must separate canonical sources from derived readable projections.
- Canonical registries must support both navigation and deterministic validation.
- Governed Markdown bodies must remain concise, analyzable and linked to their canonical registry records.
- Governed documents must use stable identifiers and canonical terminology.
- Controlled documentation fields must reference governed value sets, taxonomies or vocabularies.
- Governed documentation must distinguish authored prose, required fields, controlled fields and derived values.
- Decision records must govern specific choices within one Macro-requirement.
- Functional Requirements must describe independently testable capabilities, behaviors or outcomes.
- Specialized Requirements must attach governed constraints, controls or qualities to Functional Requirements.
- Governed assets must use stable identities and canonical descriptions when they enter the project model.
- Governed relations must connect Macro-requirements, Decisions, Requirements, assets, implementations and verification evidence explicitly.
- ThreatForge must organize tutorials, how-to guides, reference material and explanations according to Diátaxis.
- Explanation and how-to documents must not become independent canonical sources of governed obligations.
- Derived documentation must obtain indexes, linked records and metadata from canonical sources instead of manual duplication.
- Governed documentation must preserve sufficient lifecycle and historical traceability for superseded, deprecated or removed records.
- Deterministic corpus analysis must identify terminology drift, ambiguous labels and uncontrolled references.
- Governed documentation must support future graph exploration and threat-analysis workflows without duplicating canonical facts.

Scope

- Includes: Macro-requirement registry records and Markdown bodies
- Includes: Decision and Requirement registries and bodies
- Includes: [BAE-0001] Governed project documentation
- Includes: governed assets, controlled vocabularies and taxonomies
- Includes: stable identifiers, controlled fields and explicit relations
- Includes: implementation and verification traceability
- Includes: Diátaxis documentation connected to governed sources
- Includes: deterministic corpus-quality reports and derived documentation projections
- Excludes: final application backend and user-interface implementation
- Excludes: runtime threat-analysis execution
- Excludes: autonomous AI integrations and model runtime behavior
- Excludes: ungoverned narrative content without a governed project-model relation

Non-goals

- Mandatory adoption of RDF, SKOS, SHACL or OWL in the initial model
- Manual maintenance of indexes or metadata already derivable from canonical sources
- Use of explanation or how-to documents as autonomous normative sources

Proposed rewrite for metamodel test

Proposed: MR-0001 - Documentazione di progetto governata e tracciabile

Intent Consentire a persone e strumenti di comprendere, mantenere e verificare le conoscenze che guidano un progetto attraverso documentazione versionata, coerente e tracciabile.

Context ThreatForge usa la documentazione governata come base condivisa per descrivere intenzioni, scelte, requisiti e relative evidenze. La documentazione deve rimanere leggibile dalle persone e sufficientemente strutturata da poter essere verificata e navigata dagli strumenti.

Stakeholders

- responsabili di progetto
- sviluppatori e manutentori
- reviewer
- analisti
- utenti che governano progetti con ThreatForge

Objective

- Rendere la documentazione una fonte affidabile per comprendere e governare l'evoluzione del progetto.
- Mantenere espliciti i collegamenti tra intenzioni, decisioni, requisiti ed evidenze senza duplicare le stesse regole in più livelli.

Scope (candidate semantics under test)

- conoscenza governata del progetto e sua tracciabilità
- leggibilità umana e verificabilità automatizzabile della documentazione

What moved down

Registri, Markdown body, Diataxis, materializzazioni, tassonomie, controlli terminologici e meccanismi di proiezione diventano Decision/Requirement/Governance detail.

MR-0002

Current document

Original: MR-0002 - Authoring e implementazione governata

Intent Define a governed path from an existing Requirement to planned, created and verified implementation artifacts through deterministic core capabilities and thin development-environment adapters.

Context ThreatForge supports people and LLM consumers while they author governed Requirements and connect those Requirements to implementation and verification evidence. The core workflow remains independent from a specific editor, while Visual Studio Code is the first integrated authoring surface.

The repository runner, authoring tools, implementation planners, scaffolders and promotion controls form one governed workflow. Editor integrations collect input, invoke those capabilities and present their results without owning domain rules or bypassing repository gates.

Macro obligation

- Every implementation artifact must derive from at least one existing governed Requirement.
- ThreatForge must keep domain rules and traceability rules inside governed core modules and tools.
- Development-environment adapters must remain thin consumers of governed capabilities.
- Requirement authoring must use canonical registries, controlled value sets and deterministic body profiles.
- Requirement creation must provide a complete preview before persistent repository changes.
- Persistent authoring operations must require explicit confirmation.
- Governed creation operations must preserve atomicity across registry and body updates.
- Governed creation operations must restore pre-operation state after verification failure.
- Implementation planning must remain read-only before artifact creation.
- Every implementation plan must identify the governing Requirement.
- Implementation scaffolding must create traceable artifacts with controlled lifecycle state.
- Implementation promotion must require completed source content and successful verification evidence.
- Implementation artifacts must declare bidirectional traceability to their linked Requirements.

- Repository commit and push operations must execute registered materializers and read-only gates before Git staging.
- Repository projection materialization must remain deterministic, bounded and idempotent.
- Direct ungoverned Git operations must not replace the governed repository runner.
- Visual Studio Code tasks must delegate to governed commands without duplicating canonical choices or validation logic.
- Future editor adapters must consume the same governed capabilities without changing the canonical core rules.
- The ThreatForge application must orchestrate the complete workflow without coupling the model to one editor or filesystem adapter.

Scope

- Includes: governed Requirement authoring and deterministic previews
- Includes: implementation planning, scaffolding and promotion
- Includes: traceability between Requirements, source artifacts and verification evidence
- Includes: registered repository projections and governed commit-push operations
- Includes: thin Visual Studio Code integration
- Includes: future editor and application adapters over the same core capabilities
- Excludes: autonomous code generation without an existing governed Requirement
- Excludes: domain rules duplicated inside editor extensions
- Excludes: ungoverned automatic commit or push behavior
- Excludes: simultaneous initial support for every development environment

Non-goals

- Complete ThreatForge application user interface in the initial authoring workflow
- Editor-specific ownership of canonical validation or controlled values
- Creation of implementation artifacts without governed planning and traceability

Proposed rewrite for metamodel test

Proposed: MR-0002 - Sviluppo tracciabile dai requisiti

Intent Consentire al lavoro di implementazione di partire da requisiti governati e di mantenere chiaro, durante l'evoluzione del progetto, che cosa viene realizzato e con quali evidenze viene verificato.

Context ThreatForge deve aiutare chi sviluppa un progetto a passare dalla documentazione governata al lavoro implementativo senza perdere il legame con le ragioni e gli obblighi che hanno originato quel lavoro.

Stakeholders

- sviluppatori
- reviewer
- responsabili di progetto
- manutentori
- utenti che usano ThreatForge per guidare l'autoring e l'implementazione

Objective

- Mantenere tracciabile il percorso dal requisito al lavoro implementativo e alla verifica.
- Offrire assistenza coerente all'autoring e all'esecuzione del lavoro senza spostare le regole di dominio dentro uno specifico strumento di sviluppo.

Scope (candidate semantics under test)

- passaggio governato da requisiti a implementazione e verifica
- tracciabilità dello stato del lavoro rispetto ai requisiti

What moved down

VS Code, repository runner, preview, atomicity, rollback, scaffolding, promotion, Git staging e materializer sono scelte/obblighi di livello inferiore.

MR-0003

Current document

Original: MR-0003 - Modello di Analisi Base derivato dalla documentazione

Intent Define a methodology-neutral and documentation-derived system model that transforms governed project knowledge into a canonical inventory of Base Analysis Elements for threat analysis before implementation and throughout system evolution.

Context MR-0001 establishes governed documentation as the primary project model, while MR-0002 governs document authoring and the path from Requirements to implementation artifacts.

ThreatForge currently lacks an explicit analytical representation derived from governed documentation without replacing its canonical facts. Base Analysis Elements provide stable identities, provenance and relations for reconstructing the system being designed, identifying incomplete knowledge and determining whether prior analyses are stale.

This boundary supports moving threat analysis into the documentation and design phase instead of waiting for implementation artifacts to exist.

Macro obligation

- ThreatForge must maintain one canonical inventory of Base Analysis Elements for each analyzed project.
- Every Base Analysis Element must have a stable governed identity, canonical type, canonical title, explicit meaning, provenance and lifecycle state.
- Every Base Analysis Element must be justified by existing governed project knowledge or by an explicit reviewed analytical addition.
- The governed origin of a Base Analysis Element must exist before any document that references that element.
- A document must not reference a Base Analysis Element whose governed origin belongs to one of that document's descendants.
- Base Analysis Element provenance must identify the governed documents and evidence from which the element is derived.
- The Base Analysis model must preserve explicit relations among elements and between elements, Decisions, Requirements and other governed project records.
- The Base Analysis inventory must remain methodology-neutral.
- The Base Analysis inventory must not contain STRIDE, STRIDE-AI or other overlay-specific classifications as base facts.
- Threat-analysis overlays must consume the canonical Base Analysis inventory without silently adding, removing or redefining its elements.
- Missing or changed system knowledge discovered by an overlay must produce a governed proposal to update the Base Analysis inventory.
- Changes to governed source documentation must support deterministic identification of Base Analysis Elements and analyses that are potentially stale.
- The Base Analysis model must support analysis before implementation and re-analysis after architectural, requirement or feature changes.
- MR-0003 must own Base Analysis Element semantics, provenance, relations and lifecycle without owning the generic Markdown reference syntax governed by MR-0001.
- MR-0003 must expose canonical Base Analysis projections that MR-0002 authoring tools and editor adapters can consume without duplicating domain rules.

Scope

- Includes: Methodology-neutral Base Analysis model
- Includes: Canonical Base Analysis Element inventory

- Includes: Stable Base Analysis Element identities and canonical types
- Includes: Documentary provenance and governed origin
- Includes: Relations among Base Analysis Elements and governed project records
- Includes: Base Analysis Element lifecycle and staleness
- Includes: Derivation from Macro-requirements, Decisions, Requirements and other governed project documentation
- Includes: Reviewed analytical additions when governed documentation is incomplete
- Includes: Canonical system topology and data-flow projections for later threat-analysis overlays
- Includes: Traceability from source documentation to Base Analysis Elements and dependent analyses
- Excludes: Generic Markdown syntax for governed entity references
- Excludes: Editor-owned completion, hover, diagnostics or quick-fix rules
- Excludes: STRIDE and STRIDE-AI classifications
- Excludes: Methodology-specific threats, findings, mitigations or security requirements
- Excludes: Runtime Base Analysis implementation and user-interface behavior
- Excludes: Automatic canonical acceptance of LLM-inferred elements
- Excludes: Replacement of Macro-requirement, Decision or Requirement registries

Non-goals

- Freeze the complete Base Analysis Element taxonomy in the Macro-requirement
- Select the physical registry or database schema
- Implement extraction, validation, graph or editor tooling
- Treat the first textual occurrence of an element as its governed origin
- Make an LLM inference canonical without explicit governed review
- Duplicate project facts that already have an authoritative governed source

Proposed rewrite for metamodel test

Proposed: MR-0003 - Modello del progetto per l'analisi

Intent Rendere la conoscenza governata del progetto utilizzabile per analisi diverse anche prima che esista una implementazione completa, mantenendo chiara la provenienza delle informazioni e gli effetti delle modifiche.

Context Le analisi hanno bisogno di una rappresentazione coerente di ciò che il progetto descrive. Questa rappresentazione deve derivare dalla documentazione senza sostituirla i fatti canonici e deve poter evolvere quando cambia il progetto.

Stakeholders

- analisti
- architetti e progettisti
- sviluppatori
- reviewer
- responsabili di progetto

Objective

- Derivare dalla documentazione una base comune e tracciabile per l'analisi.
- Permettere analisi anticipate e ripetute durante l'evoluzione del progetto.
- Mantenere distinguibili i fatti del progetto dalle interpretazioni introdotte dalle singole analisi.

Scope (candidate semantics under test)

- conoscenza del progetto necessaria a costruire una base comune per analisi
- provenienza e aggiornamento della conoscenza analitica comune

What moved down in the first reformulation

BAE fields, precise provenance rules, topology/data-flow projection, STRIDE exclusions and editor integration move below MR.

Second reformulation after ADR-family review

Current candidate: MR-0003 - Governed representation of the system

Intent Make the system described by the project understandable through explicit, traceable and maintainable project knowledge, so that people and downstream activities can rely on a shared representation of what the system is and how that knowledge is justified.

Context Project documentation may describe relevant parts of a system across different documents and at different levels of detail. Some knowledge is stated directly, while other knowledge may need to be reconstructed, clarified or added through review.

Without a governed representation, different readers or downstream activities may interpret the same project differently, create competing identities for the same concepts, lose the origin of important knowledge, or continue relying on information that is no longer current.

Stakeholders

- Project owners and maintainers
- Architects and developers
- Reviewers
- Analysts who consume project knowledge
- Other stakeholders who need a dependable understanding of the documented system

Objective Provide one coherent and traceable representation of the system described by the project, preserving the distinction between documented knowledge, reviewed additions and later interpretations.

Scope Includes the project knowledge required to identify and understand the relevant parts of the system and their relationships.

Includes the origin, supporting evidence and evolution of that knowledge when these are necessary to determine whether it can still be relied upon.

Includes a governed way to review knowledge that is inferred, reconstructed or found to be incomplete.

Excludes interpretations whose meaning depends on a particular analysis method.

Excludes findings, threats, risk judgments and other conclusions produced by applying an analysis method.

Assumptions Project documentation may be incomplete and may evolve over time.

Some relevant system knowledge may require explicit review before it can be treated as governed project knowledge.

Constraints The governed representation must remain independent from the terminology and classifications of any particular analysis method.

Knowledge introduced through inference or analytical discovery must not become governed project knowledge without explicit review.

Why the first reformulation was reopened

The first reformulation described MR-0003 mainly through its future use by analysis (“a common base for analysis”). Read beside the first MR-0005 rewrite, this made the two MRs appear almost equivalent.

Before merging them, the historical Decision family was inspected. The MR-0003 Decisions consistently govern a different concern: representation of system knowledge, identity and provenance, continuity of source authority, documentary extraction/review, and the boundary between system facts and methodological interpretation.

The second reformulation therefore expresses the value of MR-0003 independently from its analytical consumer: **what do we know about the system, and why can that knowledge be relied upon?**

MR-0004

Current document

Original: MR-0004 - Ciclo di vita governato dei progetti target

Intent Define how ThreatForge creates, accesses, validates, isolates and analyzes governed Target Projects at user-selected filesystem locations.

Context ThreatForge needs one small and reproducible path from project creation to governed documentation and documentation-derived Base Analysis. An internal demonstration project and a newly created external project use the same Target Project model even though their destination locations differ. The ThreatForge engine remains separate from every target, and the initial demonstrable target can represent a system through governed documentation before executable application code exists.

Macro obligation

- ThreatForge must use one Target Project model for an internal demonstration project and a newly created external project.
- Target Project creation must receive one explicit destination root selected for the creation request.
- Subsequent Target Project authoring, verification and analysis must receive one explicit target root.
- Internal and external Target Projects must originate from the same governed template and application behavior.
- Every Target Project must own its project-local documentation, registries, reports and materialized projections.
- A Target Project must remain structurally valid without executable source code, a running backend, a frontend or a database instance.
- Every Target Project must support the governed document models required by the active engine-owned target template and project-local Base Analysis records.
- Target Project documentation must provide project-local sources for actors, logical components, data resources, trust boundaries and information flows.
- Target Project records must preserve stable project-local identifiers and source paths.
- Base Analysis Element provenance must resolve to governed sources owned by the analyzed Target Project or to explicit reviewed analytical additions.
- Target Project creation and use must not modify or contaminate the canonical ThreatForge project model.
- ThreatForge repository verification must remain development governance.
- ThreatForge repository verification must not be represented as a Target Project kind.
- MR-0001 must remain authoritative for governed documentation structure, document semantics and Diátaxis organization.
- MR-0002 must remain authoritative for reusable application interfaces, target-access boundaries and delivery-adaptor separation.
- MR-0003 must remain authoritative for Base Analysis Element semantics, provenance, relations and lifecycle.
- Target-specific product requirements must remain inside the governed project model of the owning Target Project.

Scope

- Includes: One Target Project model for internal and external destinations
- Includes: Explicit destination-root project creation
- Includes: Explicit target-root authoring, verification and analysis
- Includes: Shared governed Target Project template
- Includes: Project-local governed documentation and registries
- Includes: Document-only Target Projects without executable application code
- Includes: Project-local sources for actors, components, data resources, boundaries and flows
- Includes: Base Analysis readiness and documentary provenance
- Includes: Isolation of target records, reports and materializations
- Includes: Minimal Target Project lifecycle orchestration
- Excludes: Import or migration of arbitrary existing repositories
- Excludes: Compatibility-version negotiation or migration mechanisms

- Excludes: Multiple concurrent target sessions
- Excludes: Final web-interface behavior
- Excludes: Product-domain requirements of a specific Target Project
- Excludes: STRIDE, STRIDE-AI or other methodology overlays
- Excludes: Executable application code as a prerequisite for project analysis

Non-goals

- Model ThreatForge repository verification as a Target Project kind
- Define compatibility versions, gate profiles or migration mechanisms
- Import arbitrary unstructured repositories
- Implement multiple concurrent Target Project sessions
- Define the final web user interface
- Define the product-domain requirements of the first demonstration Target Project
- Require executable application code before documentation-derived Base Analysis

Proposed rewrite for metamodel test

Proposed: MR-0004 - Gestione isolata dei progetti governati

Intent Consentire a ThreatForge di creare, aprire e governare progetti distinti mantenendo separati i loro documenti, risultati e ciclo di vita dal motore ThreatForge e dagli altri progetti.

Context ThreatForge è uno strumento riutilizzabile su più progetti. Ogni progetto governato deve poter possedere la propria conoscenza e i propri risultati senza dipendere dal repository del motore o contaminare altri progetti.

Stakeholders

- utenti di ThreatForge
- team dei progetti governati
- manutentori di ThreatForge
- reviewer e analisti che lavorano su un progetto specifico

Objective

- Fornire a ogni progetto governato uno spazio autonomo e identificabile.
- Mantenere separata l'evoluzione del progetto governato dall'evoluzione del motore ThreatForge.

Scope (candidate semantics under test)

- creazione/apertura e governo di progetti distinti
- ownership project-local della documentazione e dei risultati

What moved down

Filesystem roots, templates, internal/external distinction, BAE types and adapter boundaries become lower-level choices and requirements.

MR-0005

Current document

Original: MR-0005 - Common governed analysis model

Intent Define the common model through which ThreatForge governs methodology-specific analyses based on the Base Analysis and produces repeatable derived representations.

Context The BAE registry is the methodology-neutral canonical source for the analyzed system. Transformation of BAEs into methodology-specific analysis elements involves expert judgment when the selected method does not provide an unambiguous mapping. Reviewed and accepted analysis registries provide the inputs for deterministic validation and derived representation generation.

Macro obligation

- The common analysis model must preserve the BAE registry as the methodology-neutral canonical source.
- Each analysis must record the expert-approved methodological interpretation separately.
- Methodology-specific registries must use the controlled taxonomies defined by their analysis method.
- The common analysis model must distinguish expert judgment from subsequent deterministic transformations.
- The DFD of an analysis must be derived deterministically from the governed registries accepted for that analysis.
- The renderer must consume a derived projection.
- The renderer must not contain Base Analysis or methodology-specific rules.
- Method-specific taxonomies and rules must belong to dedicated Macro-requirements.
- An analysis that identifies missing Base Analysis knowledge must record the discrepancy explicitly.
- The analysis process must not modify BAE registry records automatically.

Scope

- Includes: common model and boundaries of governed analyses
- Includes: relationship between the canonical Base Analysis and methodology-specific registries
- Includes: separation between expert interpretation and deterministic transformations
- Includes: deterministic DFD derivation from accepted analysis registries
- Includes: separation between DFD projection and rendering
- Excludes: STRIDE-specific categories and rules
- Excludes: STRIDE-AI-specific categories and rules
- Excludes: detailed definition of DFD detail levels
- Excludes: automatic promotion of analysis discoveries into the BAE registry
- Excludes: automatic replacement of expert judgment

Non-goals

- Define the complete registry schemas for every methodology
- Define detailed diagram layout rules
- Implement the complete correction lifecycle for elements missing from the Base Analysis

Proposed rewrite for metamodel test

Proposed: MR-0005 - Analisi governate e ripetibili

Intent Consentire di applicare analisi differenti allo stesso progetto mantenendo espliciti il giudizio esperto, la provenienza dei risultati e le parti che possono essere ripetute deterministicamente.

Context Uno stesso progetto può essere osservato con paradigmi e metodi differenti. ThreatForge deve permettere queste interpretazioni senza alterare implicitamente i fatti comuni del progetto e senza confondere risultati metodologici con conoscenza di base.

Stakeholders

- analisti
- reviewer
- responsabili di progetto
- specialisti delle discipline di analisi
- utenti che confrontano o rieseguoano analisi

Objective

- Governare analisi metodologicamente differenti sopra una base comune del progetto.
- Rendere tracciabili le interpretazioni e i risultati prodotti da ciascuna analisi.
- Permettere di ripetere in modo deterministico le trasformazioni che non richiedono nuovo giudizio esperto.

Scope (candidate semantics under test)

- ciclo comune delle analisi e loro tracciabilità
- separazione tra conoscenza comune, interpretazione metodologica e risultati derivati

What moved down in the first reformulation

DFD, renderer, method taxonomies, plugin specifics and exact registry shapes are lower-level analysis/model decisions, not MR obligations.

Second reformulation after ADR-family review

Current candidate: MR-0005 - Governed analysis of the system

Intent Enable different analysis methods to examine governed project knowledge and produce reviewable, traceable and repeatable results without changing the underlying description of the system or hiding method-specific interpretation.

Context A governed description of the system provides shared knowledge about what is being developed, but different analysis methods can interpret that knowledge from different perspectives and may require expert judgment.

Without a governed analysis boundary, analytical interpretations may be confused with project facts, results from different methods may become difficult to compare or trace, and conclusions may lose their connection to the project knowledge and requirements that motivated them.

Stakeholders

- Analysts
- Security and assurance specialists
- Project owners and maintainers
- Architects and developers
- Reviewers responsible for evaluating analysis results

Objective Allow one governed description of a system to support multiple analysis methods while preserving the method used, the analyst's interpretation, the evidence considered and the results produced.

Scope Includes governed applications of analysis methods to project knowledge.

Includes explicit analytical interpretation and expert judgment.

Includes traceability from analysis results to the project knowledge and requirements they concern.

Includes review of analytical results and feedback when analysis reveals missing, ambiguous or inconsistent project knowledge.

Excludes modification of governed project knowledge merely because an analysis method interprets it differently.

Excludes the internal taxonomy, rules or procedure of any specific analysis methodology from the common analysis model.

Assumptions Different analysis methods may interpret the same project knowledge differently.

Some analysis activities require expert judgment and cannot be reduced entirely to deterministic transformation.

Constraints Method-specific interpretation must remain distinguishable from governed project knowledge. Analysis results must retain enough provenance to identify the method, relevant project knowledge and supporting analytical evidence.

Discoveries about missing or incorrect project knowledge must return through the governed documentation process rather than silently changing canonical project knowledge.

Why the first reformulation was reopened

The first reformulation described MR-0005 as applying different analyses over a common project base. That was correct but too close to the first MR-0003 rewrite, which had also been framed around enabling analysis.

Inspection of the historical MR-0005 Decisions showed a coherent second concern: expert analysis records, common findings and documentation feedback, methodology-specific extension boundaries, and governed use of accepted findings.

The second reformulation therefore centers MR-0005 on a different question: **what do we learn by analysing the governed system knowledge, and how do we govern that interpretation and its results?**

Lesson learned from MR-0003 versus MR-0005

Observation

The first rewrites made MR-0003 and MR-0005 look almost duplicative. A merge initially appeared plausible. That conclusion was deliberately suspended and the historical Decision families were inspected before changing the MR decomposition.

Evidence that preserved the distinction for further testing

- MR-0003 Decisions form a coherent family around **governed system knowledge**: representation, identity/provenance, source continuity, documentary extraction/review and the semantic boundary between facts and method-specific interpretation.
- MR-0005 Decisions form a coherent family around **governed analytical interpretation and results**: expert analysis records, findings/feedback, methodology extension boundaries and eligibility of accepted results for downstream use.
- The distinction survives removal of current solution names such as BAE, Analysis Record, Finding, DFD and plugin.
- A dependency can be described without hierarchy: governed analysis consumes governed system knowledge, but consuming it does not make analysis the same concern as representing it.

Questions that must be asked before merging or splitting Macro Requirements

1. **Independent value**: does each candidate MR express a project-level result that remains meaningful without describing the other?
2. **Decision-family coherence**: do the descendant Decisions form distinct, internally coherent families of choices?
3. **Solution-vocabulary removal**: does the distinction remain after removing names of current components, schemas, models, tools and other chosen mechanisms?
4. **Input versus result**: is one concern about establishing governed knowledge while another is about interpreting or transforming that knowledge? If so, is that distinction meaningful to the project rather than merely an implementation pipeline?
5. **Dependency versus containment**: can the relation be expressed as `dependsOn` rather than by merging the concerns or making one a child of the other?
6. **Stakeholder explanation**: can a stakeholder competent in the project domain explain the difference using title + Intent without knowing the implementation?
7. **Merge stress test**: if the MRs were merged, would the resulting MR need to explain two substantially different families of Decisions or two different kinds of project value?
8. **Split stress test**: if they remain separate, is the separation justified by project concerns, or only by the fact that the current architecture has two layers/subsystems?

Current disposition

No merge decision is taken. The current working hypothesis keeps MR-0003 and MR-0005 distinct because the historical Decision families reveal two different semantic concerns. This remains a case-study result, not a universal rule that every project must separate system representation from analysis.

General lesson for the metamodel

When refactoring an existing governed corpus, similarity between rewritten MR prose is evidence to investigate, not sufficient evidence to merge. The candidate MR boundary must also be tested against the semantic responsibility of its descendant Decisions and against a version of the concern stripped of current solution vocabulary.
